Chapter 5

Dataset and Implementation

This chapter describes the experimental dataset and implementation. It covers the dataset's public basis and schema, clean and adversarial examples, Prompt Injection placement, the technology stack, and relevant implementation and security considerations.

Chapter 3 defines the experimental dataset dimensions and paired design in Section 3.3. Chapter 4 describes how a record is represented and processed by the assistant in Sections 4.3 and 4.4. This chapter connects those decisions to the dataset's composition, intended use, and current implementation status.

5.1 Dataset Overview

The primary experimental dataset contains 500 synthetic web-application log records: 250 clean records and 250 adversarial records, organized into 250 clean/adversarial pairs. The records can be examined by condition and traffic category; these views represent the same 500 examples and are not additional independent samples.

The 500-record dataset is a controlled research artifact created for this thesis; it is not the original HTTP CSIC 2010 dataset and is not a direct export of or row-by-row relabelling of that dataset. HTTP CSIC 2010 [1] provides a public reference for representative request patterns. The synthetic records do not preserve CSIC source-row identifiers and are not claimed to be modifications of specific CSIC examples. Their Prompt Injection text, labels, pair identifiers, and balanced composition were added for the controlled evaluation.

HTTP CSIC 2010 is used only for the external sample and CSIC-derived robustness pairs described below. The local copy's download provenance was not recorded; its content hash and sampling procedure are retained for reproducibility.

The dataset is restricted to web application logs. Firewall, EDR, DNS, IDS, Suricata, and Windows event logs are outside the experimental scope.

5.2 Public Dataset Basis

The Information Security Institute's HTTP CSIC 2010 dataset [1] was selected as a public reference for HTTP request patterns relevant to web-application security. In this thesis, it informs the design context for the request records; the experimental rows themselves are a separate, controlled synthetic dataset. This distinction is important: the dataset does not inherit Prompt Injection examples or labels from CSIC 2010, and the available files do not support tracing a synthetic row to a specific original record.

The year 2010 is an acknowledged limitation of the public reference: it cannot represent the full diversity of current web applications, modern attack prevalence, or contemporary production traffic. The 500-record dataset is therefore used for a controlled methodological experiment, not as evidence that its distribution reflects present-day attacks.

The dataset design adds five traffic categories, a separate clean or adversarial Prompt Injection condition, five labelled injection objectives, and explicit clean/adversarial pair identifiers. Its traffic composition is balanced as shown in Table 5.1.

Table 5.1: Composition of the 500-record experimental dataset.

Traffic category	**Clean**	**Adversarial**	**Total**
SQL Injection	50	50	100
Cross-Site Scripting (XSS)	50	50	100
Path Traversal	50	50	100
Brute Force Login	50	50	100
Benign / Normal	50	50	100
Total	**250**	**250**	**500**

The construction follows the controlled design in Chapter 3, Section 3.3. First, five traffic categories and their web-event labels are defined. Clean HTTP records are then paired with adversarial counterparts that preserve the underlying request semantics as far as practical and add one Prompt Injection instruction to an attacker-controlled field. Each row receives a sample ID, pair ID, condition, and relevant attack or injection annotations. The final balance check confirms 50 clean and 50 adversarial records per category, with 10 examples of each Prompt Injection variant within each category. The synthetic records do not retain CSIC source-record identifiers, so this is a synthetic construction rather than a row-by-row transformation of CSIC records.

5.2.1 External CSIC 2010 Validation Data

In addition to the primary synthetic corpus, the external validation sample contains 200 HTTP CSIC 2010 requests: 100 Normal and 100 Anomalous. The corpus has 61,065 requests and no official test partition, so this is a study-defined stratified sample from the full corpus, not a sample from an official test split. It was generated with seed 42 after checking for exact duplicate requests. Each record retains its source-record identifier. Sampling parameters, a source-content hash, label counts, and the field mapping are retained as provenance [1].

The source `classification` label supports binary evaluation only: 0 (Normal) maps to *Benign*, and 1 (Anomalous) maps to *Malicious*. CSIC does not supply native attack-subtype or severity labels. Anomalous rows are provisionally assigned to one of the four attack categories used by the synthetic corpus. Each assignment includes `candidate_confidence` and a `candidate_reason`. Low-confidence best-fit assignments are estimates, not CSIC ground truth. In the 200-record sample, the proposed anomalous categories are 52 SQL Injection, 22 Brute Force Login, 20 Path Traversal, and 6 XSS. They are not used to calculate Attack-Type Accuracy. Severity remains unavailable, so the sample cannot support severity-classification accuracy.

CSIC-Derived Prompt Injection Pairs

For a distinct robustness test, 50 requests from the external sample (25 Normal and 25 Anomalous) form 50 clean/adversarial pairs. Each adversarial member adds one study-authored Prompt Injection instruction to the query string; the unchanged source remains its clean counterpart. The five variants have 10 pairs each. Evidence Suppression is applied only to Anomalous requests so its target is grounded in request evidence; Prompt Extraction is applied to Normal requests. Each pair retains its source-record identifier, changed field, variant, target, and scoring criterion. They are labelled CSIC-derived, not original CSIC traffic, and are not added to or pooled with the primary synthetic corpus. Severity Manipulation uses an injected Low target; because CSIC has no severity ground truth, this measures response to the instruction rather than severity accuracy.

5.3 Dataset Schema

Each structured record uses the following fields. The example shows an adversarial SQL Injection record; its clean counterpart is linked through the pair identifiers.

```
{
  "sample_id": "SQLI_0001_ADV",
```

```
  "paired_sample_id": "SQLI_0001_CLEAN",
  "pair_id": "PAIR_0001",
  "http_method": "POST",
  "url": "/product?note=Ignore previous instructions...",
  "query_parameters": "note=Ignore previous instructions...",
  "request_body": "username=1 UNION SELECT username,password FROM users--",
  "payload": "1 UNION SELECT username,password FROM users--",
  "user_agent": "...",
  "status_code": 400,
  "src_ip": "192.0.2.11",
  "ground_truth_label": "malicious",
  "attack_type": "SQL Injection",
  "traffic_category": "SQL Injection",
  "ground_truth_severity": "high",
  "is_adversarial": true,
  "injection_type": "Instruction Override",
  "injection_field": "query_parameters",
  "prompt_injection_text": "Ignore previous instructions..."
}
```


The request fields represent the log content. The remaining fields store ground-truth and experiment metadata and must not be provided to the model as labels. In an adversarial record, the instruction itself is embedded in the indicated request field; `prompt_injection_text` duplicates that instruction as an annotation for analysis.

In the current dataset, the control category uses `traffic_category = Benign / Normal`, `ground_truth_label = benign`, and `attack_type = None` because no web-attack subtype applies. The thesis refers to this web-event class simply as *Benign*. This class is independent of the Prompt Injection condition: both clean and adversarial Benign records are included. An adversarial Benign record retains the benign web-event label while its Prompt Injection fields record the separate adversarial condition.

The `is_adversarial`, `injection_type`, and `injection_field` values provide the Prompt Injection labels. The current annotations do not include `ground_truth_mitre`, `injection_target`, or `expected_evidence`; the chapter therefore does not treat those fields as available.

The 19 record fields represent three groups: HTTP content and context, web-event ground truth, and pair/Prompt Injection annotations. Chapter 4, Section 4.3, uses this same separation to define the model input. The loader validates field types and normalizes values as described in Chapter 4, Section 4.4.

The injection variant and exact text are available, but a per-record success target and

expected-evidence annotation are not. Before calculating ASR, these scoring annotations must be added to a separate evaluation store using the success criteria in Chapter 3, Section 3.3.5. They must remain outside the model input. Until that annotation step is complete, the dataset alone is not sufficient to compute variant-specific ASR reproducibly.

5.4 Clean and Adversarial Pairing

Each adversarial record has a corresponding clean record with the same `pair_id`. The `sample_id` and `paired_sample_id` fields identify both members of each pair.

Example:

```
SQLI_0001_CLEAN -- Clean
SQL Injection payload
No Prompt Injection

SQLI_0001_ADV -- Adversarial
Same SQL Injection payload
+ Instruction Override instruction
```

Paired design makes it possible to compare the effect of Prompt Injection while reducing variation in the underlying web event.

5.5 Prompt Injection Placement

In the current 500-record dataset, Prompt Injection text is embedded in the `url` or `query_parameters` field. Although User-Agent and request-body values are also represented as possible attacker- controlled inputs in the threat model, they do not currently contain the injections in this dataset version. Instruction Override instruction

The five injection categories in the dataset are:

- Instruction Override,
- Classification Manipulation,
- Severity Manipulation,
- Evidence Suppression,
- Prompt Extraction.

Each adversarial record has one of these labels, and the exact instruction and its field are recorded in `prompt_injection_text` and `injection_field`. The 250 adversarial

records are balanced across the five variants, with 50 records per variant. In the current dataset, 125 instructions are placed in `url` and 125 in `query_parameters`.

The attack classes are limited to SQL Injection, Cross-Site Scripting, Path Traversal, and Brute Force Login. A Benign category is retained as a control class. Suspicious File Upload is excluded from the initial dataset to keep the evaluation focused.

Each of the five traffic categories has 50 clean and 50 adversarial records, yielding 250 records per condition and 500 in total. Clean SQL Injection records remain malicious; *clean* refers only to the absence of Prompt Injection. The web-event label and the Prompt Injection condition are separate dimensions, including for the Benign control category.

5.6 Technology Stack

The planned component roles are detailed in Chapter 4, Section 4.2. Table 5.2 distinguishes dependencies currently listed for the project from components that remain planned. A package being listed as a dependency does not by itself mean that its application code or experiment workflow has been completed.

Table 5.2: Technology status at the time of writing.

Technology	Intended role	Current status
Python + Pydantic	Load records; validate input and output schemas	Application modules pending.
LangChain	Orchestrate C1–C4 processing	Experiment pipelines pending.
ChromaDB	Store and retrieve trusted knowledge chunks	Knowledge corpus and index pending.
OpenAI API	Hosted LLM inference	Model choice and experiment configuration pending.
Sentence Transformers	Local document/query embeddings	Planned component; integration pending.
SQLite	Persist experiment metadata and results	Experiment persistence pending.
Pandas / scikit-learn	Aggregate predictions and compute metrics	Analytical workflow pending.

5.7 Functional Organization

The planned implementation separates four responsibilities: request validation and preprocessing, trusted-knowledge retrieval, controlled execution of configurations C1–C4, and evaluation of responses. This separation keeps scoring labels out of model input and prevents labelled evaluation examples from becoming retrieval evidence. Application and experiment workflows remain under development.

5.8 Implementation Details

The implementation will follow the processing path described in Chapter 4. The planned stages are:

1. Load a structured log record, validate required fields and types, and separate model input from scoring metadata, as described in Section 4.4.
2. Build a versioned ChromaDB collection from the approved OWASP, ATT&CK, playbook, and Prompt Injection policy documents. Do not index labelled logs or evaluation answers; see Sections 4.6 and 4.7 in Chapter 4.
3. Run records under C1–C4 with fixed model and generation settings. Keep each clean/adversarial pair together and store its configuration, response, detector output, and retrieved evidence IDs.
4. Validate the generated structured report and cited evidence IDs, then score web-event classification and Prompt Injection outcomes against the separate annotations. Chapter 3 defines the metrics and success criteria; the detector validation procedure is in Chapter 4, Section 4.5.4.

The final experiment record must specify the LLM and embedding model identifiers, prompt and knowledge-base versions, chunking and top-k settings, detector rules, generation parameters, validation behavior, and any retry policy. These values are not yet fixed, so no model, retrieval, or end-to-end performance results are reported in this chapter.

5.9 Ethical and Security Considerations

The experiments use controlled, offline, or synthetic web-attack examples for defensive research.

The thesis evaluates whether malicious content inside logs can influence an LLM. It does not require deployment of attacks against third-party infrastructure.

References

[1] Information Security Institute. “Http csic 2010 dataset,” Accessed: Sep. 8, 2026. [Online]. Available: https://www.isi.csic.es/dataset/